2013年

进程映象

试写出进程映象包括哪些组成部分（不必项数每个部分的具体内容）

- 进程控制快：进程的标识信息、状态信息和控制信息
- 进程程序块：进程执行的程序空间
- 进程数据块：进程处理的数据间，包括数据、处理函数的用户栈和可修改程序
- 核心栈：在内核模式下运行时使用的堆栈，中断或系统过程使用

磁盘调度（循环扫描）

假定磁盘有 200 个柱面，编号 0~199，当前存取臂的位置在 100 号柱面上，并刚刚完成了 80 号柱面的服务请求，如果请求队列的先后顺序是：55、58、39、18，90，160、150、38、184；试问：如果采用**循环扫描**算法完成上述请求，其存取臂移动的总量是多少？并写出磁头臂移动的序列。

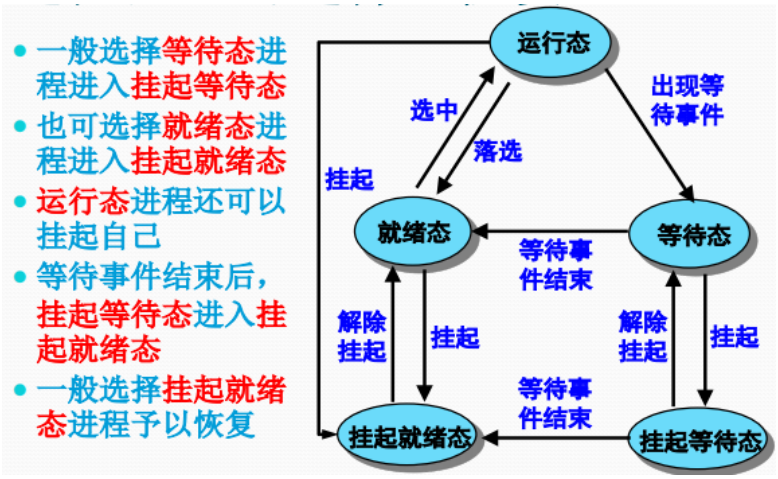
- 循环扫描，先确定继续前进的方向是向大号（因为80->100），所以序列是150-160-184-199-0-18-38-39-55-58-90，移动总量 (199-100) + 199 + 90 = 388

间接盘块计算

UNIX 系统中，每个 i 节点中分别含有 12 个直接地址的索引和一、二、三级间接索引。假设每个盘块 1024Byte，若每个盘块放 256 个盘块地址，25MB 的文件分别占用多少直接、一、二、三级间接盘块?

- 直接块容量：12 * 1024B = 12 KB
- 一级间接容量：256 * 1024B = 256 KB
- 二级间接容量：256 * 256 * 1024B = 2^{16} KB
- 25 MB = 25 * 2^{10} KB < 2^{16} KB，所以用到二级间接盘块
- (25 * 2^{10} - 12 - 256) / 1KB = 25332 个，所以用到25332个二级间接盘块，256个一级间接盘块，12个直接盘块

五状态进程图及其状态转换图



反置页表计算

一台机器有 48 位虚地址和 32 位物理地址，若页长为 8KB，问页表共有多少个页表项？如果设计一个反置页表，则有多少个页表项?

- 因为页长 8KB 占用 13 位，所以，页表项有 2^{35} 个。反置页表项有 2^{19} 个

2014

间接盘块计算

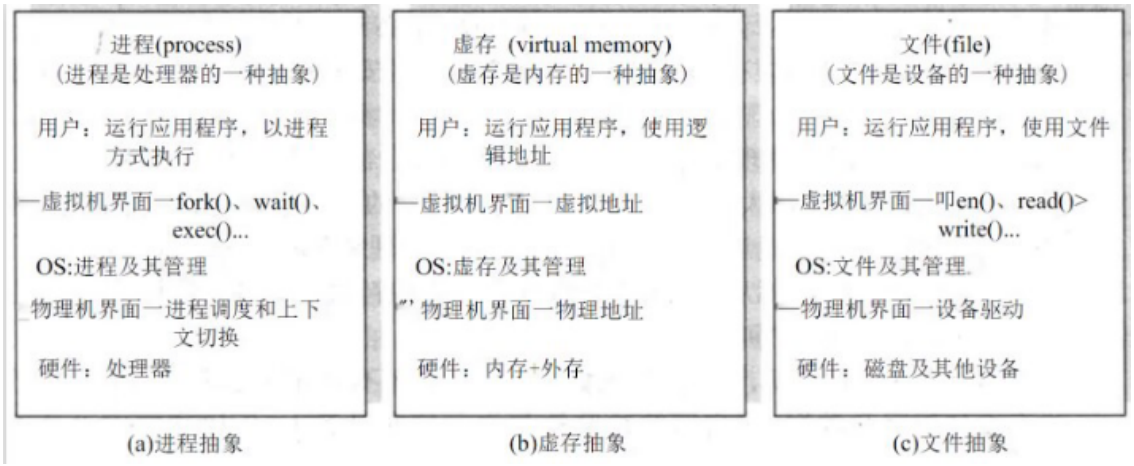
同上（数据变了，方法一样）

磁盘调度算法（扫描、电梯）

有一具有 40 个柱面的盘面，编号为 0~39，当磁头位于第 11 磁道时，顺序到来如下柱面请求：柱面号：1、36、16、34、9、12；试用如下磁盘驱动调度算法计算磁头臂移动的总量是多少？并写出磁头臂移动的序列。(1) 扫描算法 SCAN，(2) 电梯调度。

基础抽象

试述操作系统中三个最基础的抽象，并回答为什么要引入它们?

- 进程抽象、虚存抽象、文件抽象
- 

(a)进程抽象　(b)虚存抽象　(c)文件抽象
- 为了方便对物理资源的管理和控制

2015

间接盘块计算

同上

进程映象组成部分

同2013年

磁盘调度算法（电梯）

基本一样的算法（看大题整理）

七状态进程图及其状态转换图

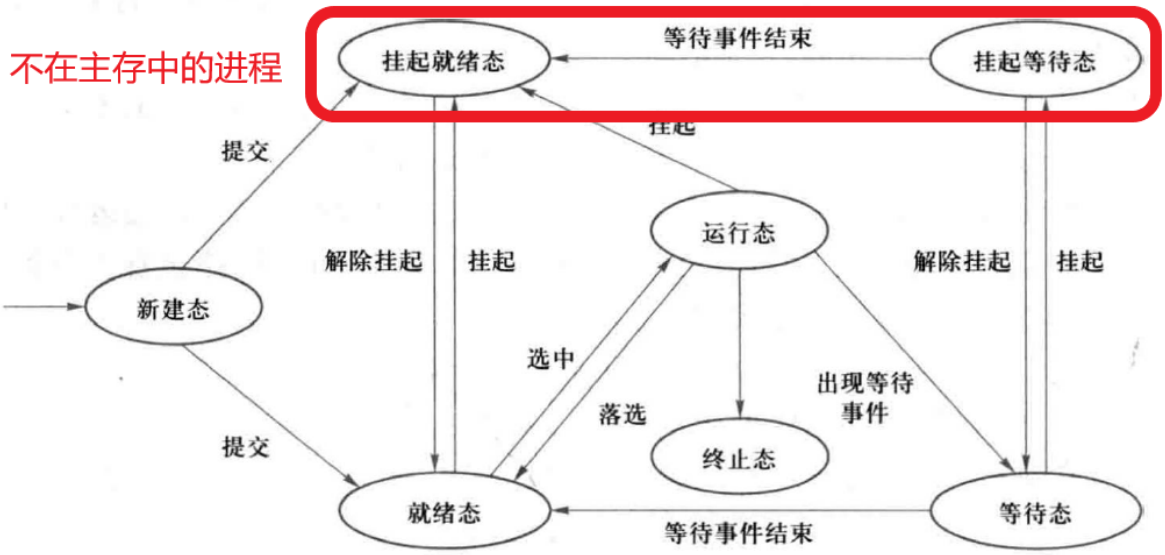


图 2-5　具有挂起功能的系统的进程状态及其转换示意图

2016

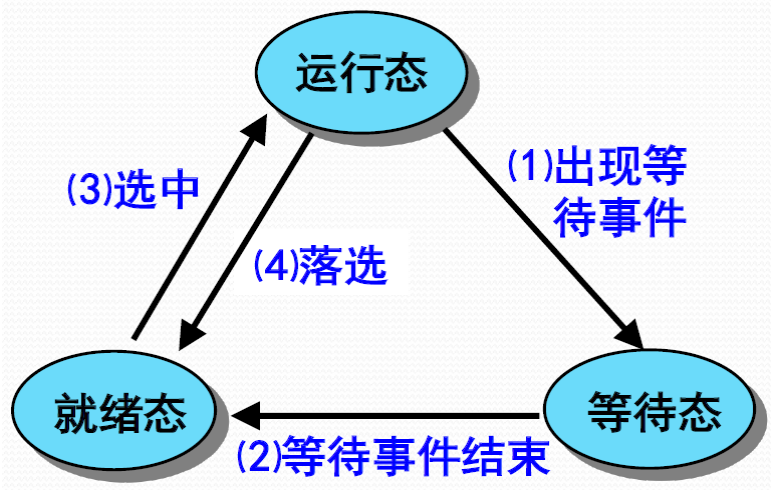
进程映象组成部分

同2013

磁盘调度算法（电梯）

基本一样的算法（看大题整理）

三状态进程图及其状态转换图



间接盘块计算

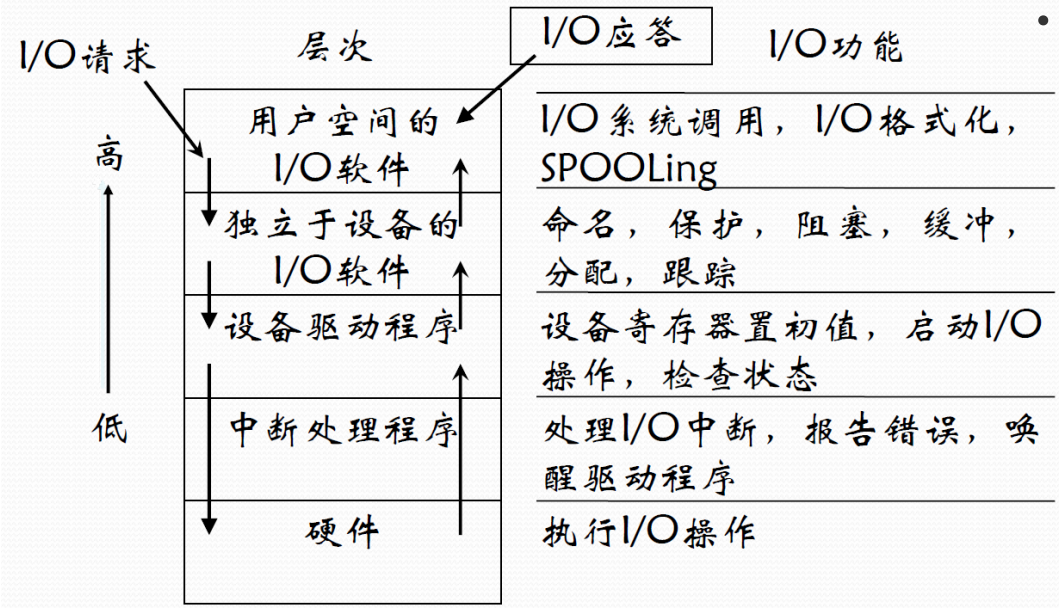
同上

2019

IO软件层次

自顶向下简述IO软件的四个层次

-
- 用户空间的IO软件：是在用户态运行的应用程序或库函数



- 独立于设备的IO软件：是在内核态运行的操作系统组件，提供了与设备无关的I/O接口，屏蔽了不同设备之间的差异（通过路径命名寻址设备、进设备保护、缓冲技术、设备分配和状态跟踪）
- 设备驱动程序：是在内核态运行的操作系统组件，从独立于设备的软件中接收并执行 I/O 请求
- 中断处理程序：在内核态运行的操作系统组件，负责响应设备控制器发出的中断信号，执行中断服务例程，通知设备驱动程序或设备独立性软件完成I/O操作

磁盘调度算法（电梯）

基本一样的算法（看大题整理）

fork()进程数，画出进程树

三次fork，多七个进程，斐波那契（是真斐波那契，不是说卷子，虽然卷子也）

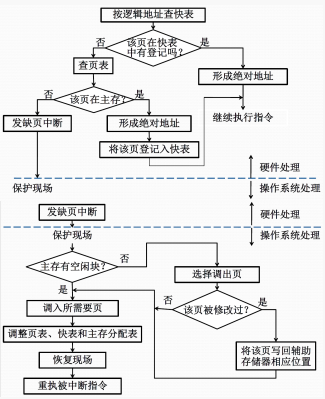
间接盘块计算

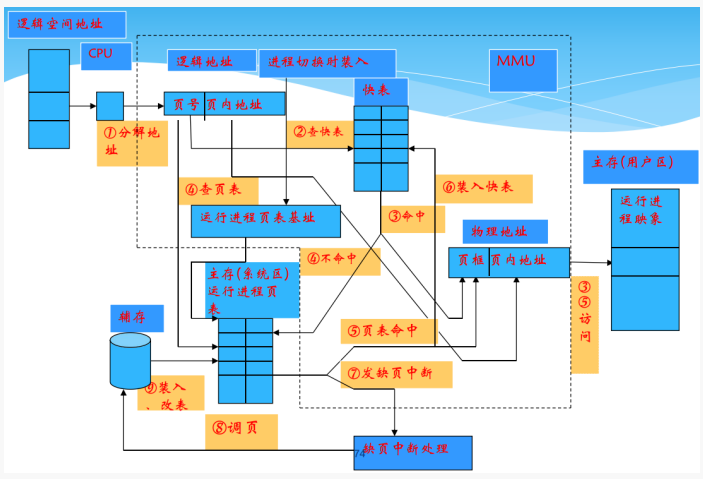
同上

虚存分页

简述虚存分页的原理，并画出流程图

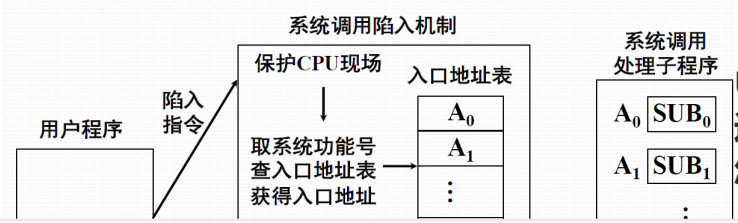
- 将进程信息副本存放在外存中，当它被调度投入运行时，程序和数据没有全部装入内存，仅装入当前使用页面，进程执行过程中访问到不在内存的页面时，再由系统自动调入。





2020

简述操作系统的系统调用，并说明陷入机制以及其流程

- 系统调用是指用户程序请求操作系统内核提供某种服务或执行某些特权操作的过程
- 陷入处理机制是计算机系统中执行和实现系统调用的机制
  - CPU 执行程序中编写的由访管指令实现的系统调用时会产生异常，通过陷阱机制，处理器的状态由用户态转换为内核态，进入操作系统并执行相应的服务例程，以获得操作系统服务。
  - 

Unix的open参数以及返回值，在操作系统中怎么进行系统调用

```c
int fd, mode;
char * filenamep;
fd = open
(filenamep,mode);
```

- 参数为文件名、设备类(号)、打开方式
  - `char *fienamep` 是指向要打开的文件路径名的字符串指针
  - `int mode` 是文件所具有的权限
- 返回值为 `int fd`，是文件打开成功后系统返回的文件描述符
- 处理流程：
  - 检索目录，把它的**外存索引节点复制到活动索引节点表**
  - 根据参数mode核对权限，如果非法，则这次打开失败
  - 当"打开"合法时
    - 为文件分配**用户打开文件表项和系统打开文件表项**，并为表项设置初值
    - 通过**指针建立这些表项与活动索引节点间的联系**
    - 把文件描述字，即**用户打开文件表中相应文件表项的序号返回给调用者**

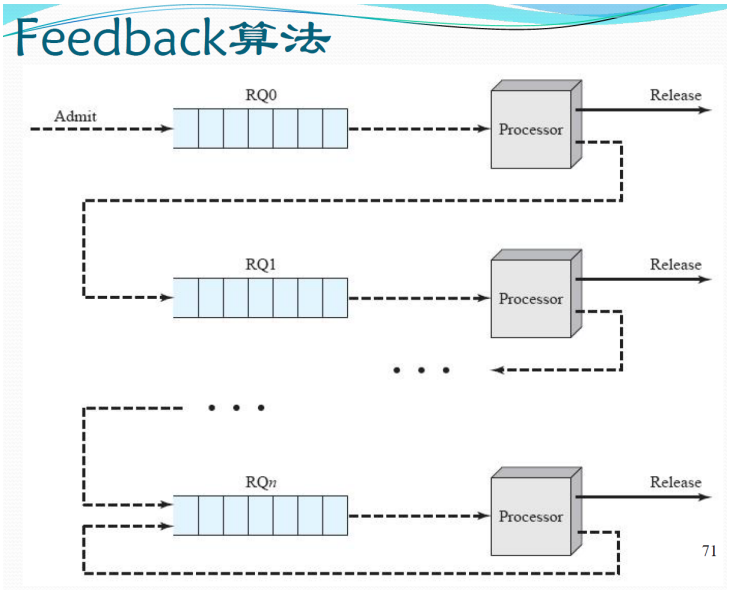
根据进程状态模型解释操作系统为何是中断驱动的，这里的中断是广义的中断

- 操作系统是**中断驱动**的；换言之，**中断是激活操作系统的唯一方式。**
- 可以结合进程其状态解释一下（？）

2021

feedback

feedback模型图、工作原理、与rr相比的优势；feedback的缺点和改进措施

- 

- 工作原理：
  - 建立多个不同优先级的就绪进程队列；
  - 多个就绪进程队列之间按照优先级调度；
  - 高优先级的就绪进程，分配的时间片短；
  - 单个就绪进程队列中的进程的优先级和时间片相同，按照先来先服务算法调度。
- 与rr相比的优势
  - 在RR算法中，只有一个就绪队列， 不分优先级，RR算法延长了短进程的等待时间
- 缺点与改进
  - 对以计算为主的长进程不友好，就绪队列越深获得调度的机会越小——改进：不同进程时间片不同，越深分配时间片越长
  - 或者可以定时刷新队列中的优先级，就是不要让他一直沉着，不然会饿死

2022

feedback

同上

Unix的create参数以及返回值，在操作系统中怎么进行系统调用

- ```
  int fd, mode;
  char *filenamep;
  fd = create (filenamep, mode);
  ```
- 参数和read一致，为文件名、设备类(号)、打开方式
  - `char *fienamep` 是指向要创建的文件路径名的字符串指针
  - `int mode` 是文件所具有的权限
- 执行过程
  - 为新文件分配索引节点和活动索引节点，并把索引节点编 号与文件分量名组成新目录项，记到目录中。
  - 在新文件所对应的活动索引节点中置初值，如置存取权限 i_mode，连接计数i_nlink等
  - 分配用户打开文件表项和系统打开文件表项，置表项初值， 读写位移f_offset清“0”
  - 把各表项及文件对应的活动索引节点用指针连接起来，把 文件描述字返回给调用者

2023
  ```

磁盘调度（电梯）

同上

进程七态

同上

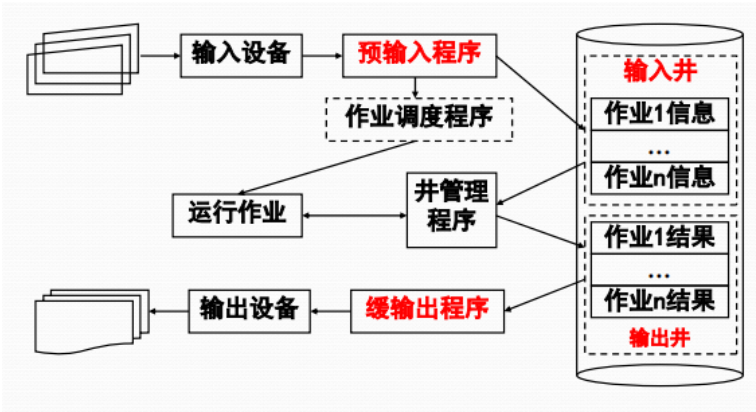
进程映象

同上

间接盘块计算

同上

SPOOLing系统结构图



- SPOOLing（外部设备联机并行操作），是关于慢速字符设备如何与计算机主机交换信息的一种技术，“假脱机”技术
- 用高速磁盘设备来模拟慢速的字符设备，缩短进程在内存中的驻留时间
    - 进程运行过程中只从输入井读入数据，只向输出井输出信息，使得全部I/O都基于磁盘
    - 预输入程序：预先把数据从输入设备传送到磁盘输入井
    - 缓输出程序：把数据从磁盘输出井传送到输出设备
    - 井管理程序：控制作进程和井之间的数据交换（事实上是I/O重定向）

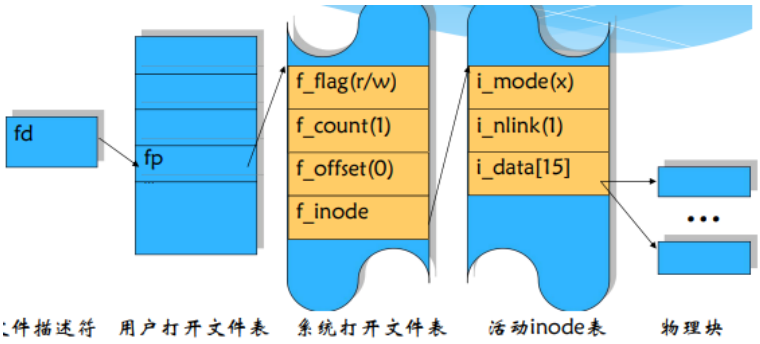
补充

文件系统

- 上面已经涉及到了creat和open，对其余操作进行一下补充
- `f_count`与`i_count`分别反映了进程动态共享一个文件的两种方式
    - `f_count`反映不同进程通过同一个系统打开文件表项共享一个文件的情况
    - `i_count`反映不同进程通过不同系统打开文件表项共享一个文件的情况
    - 通过两种方式，进程之间既可以用相同位移指针`f_offset`，也可不同位移指针共享同一个文件
- 删除
    - ```
      char* filenamep;
      unlink(filenamep);
      ```
      ```

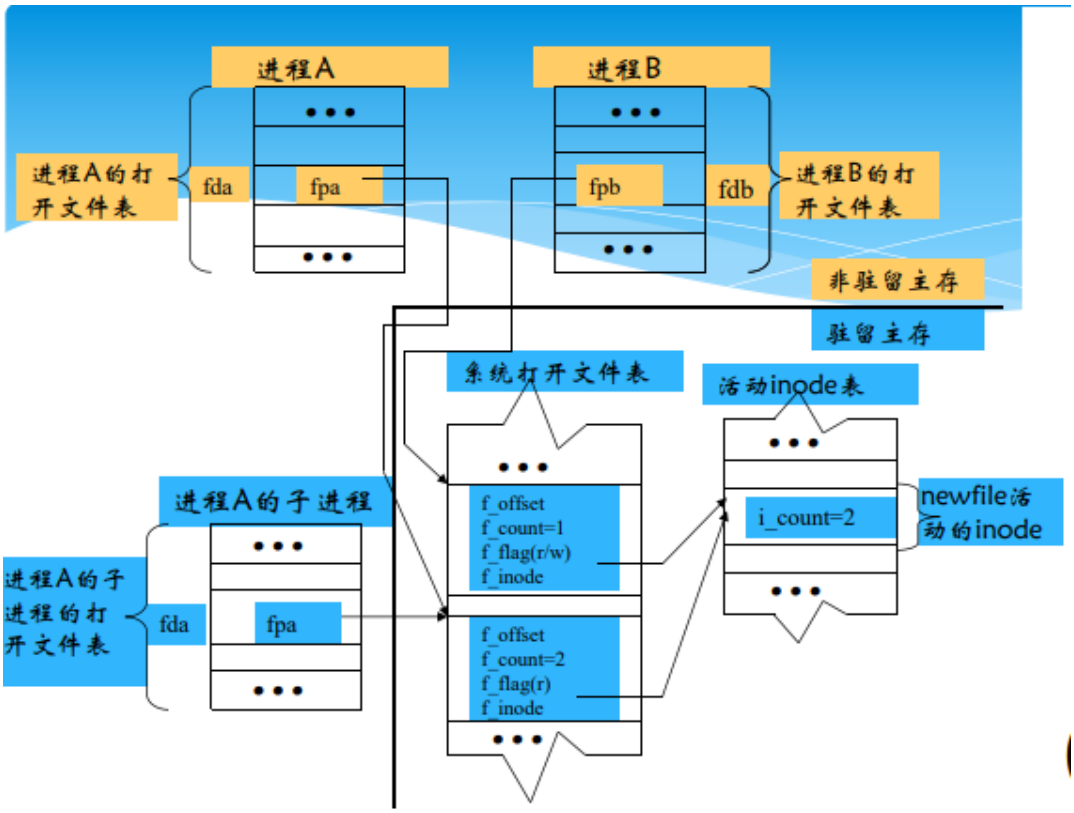
- 执行删除时，必须要求用户对该文件具有“写” 操作权

- 关闭
  - ```
    int fd;
    close(fd);
    ```
  - 参数fd是文件描述符
  - 过程：
    - 根据fd找到用户打开文件表项，再找到系统打开文件表 项。释放用户打开文件表项。
    - 把对应系统打开文件表项中的f_count减“1”，如果非 “0”，说明还有进程共享这一表项，不用释放直接返回； 否则释放表项。
    - 把活动索引节点中的i_count减“1”
      - 若不为“0”，表明 还有用户进程正在使用该文件，不用释放而直接返回
      - 否则在把该活动索引节点中的内容复制回文件卷上的相 应索引节点中后，释放该活动索引节点

- 读文件
  - ```
    int nr, fd, count;
    char buf []
    nr = read (fd, buf, count);
    ```
  - 系统根据f_flag中的信息，检查读操作合法性
  - 再根据当前位移量f_offset值，要求读出的字节数， 及活动索引节点中i_data[15]指出的文件物理块存放地址，把相应的物理块读到缓冲区中，然后再送到bufp指向的用户主存区中
  - 

- 写文件
  - ```
    nw = write (fd, buf, count);
    ```
  - 参数：`buf` 是信息传送的源地址，即把buf所指向的用户主存区信息写入到文件中

- 随机存取
  - ```
    long lseek;
    long offset;
    int whence, fd;
    lseek (fd, offset,whence);
    ```
  - 在文件初次“打开”时，文件的位移量f_offset清空为零， 以后的文件读写操作总是根据offset的当前值，顺序地读写 文件。为了支持文件的随机访问，提供系统调用lseek，它 允许用户在读、写文件前，事先改变f_offset的指向
  - 文件描述字fd必须指向一个用读或写方式打开的文件

    - 当whence是“0”时，则f_offset被置为offset
    - 当whence是“1”时，则f_offset被置为文件当前位置 加上offset
- 文件的动态共享
    - 多用户共享文件，每个希望独立地读、写文件，这时不能只 设置一个读写位移指针，须为每个用户进程分别设置一个读、 写位移指针。
    - 位移指针应放在每个进程用户打开文件表的表目中。这样， 当一个进程读、写文件，并修改位移指针时，另一个进程的 位移指针不会随之改变，从而，使两个进程能独立地访问同 一文件
    - 

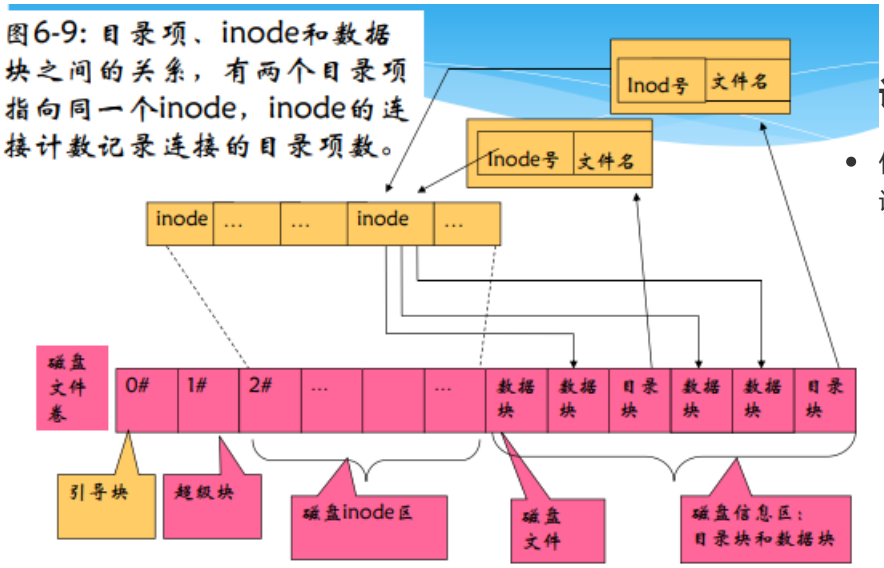
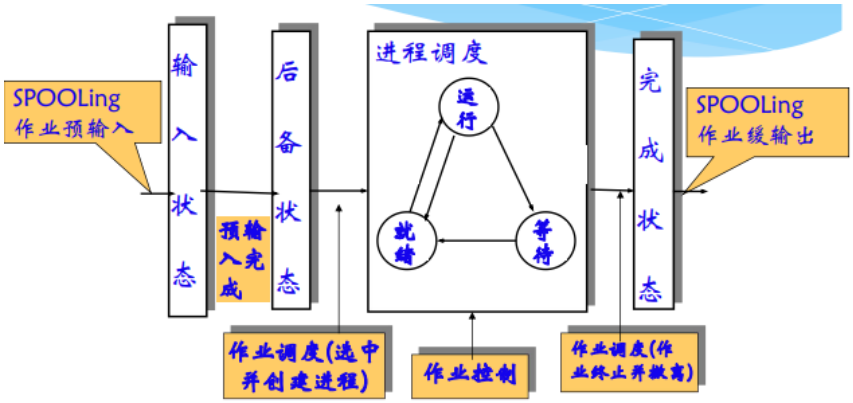
使用不同位移指针的文件共享
- 目录项、inode和数据块的关系
    - 目录项：包括文件名和inode节点号。
    - Inode：又称文件索引节点，包含文件的基础信息以及数据块的指针。
    - 数据块：包含文件的具体内容。
    - 

图6-9：目录项、inode和数据块之间的关系，有两个目录项指向同一个inode，inode的连接计数记录连接的目录项数。

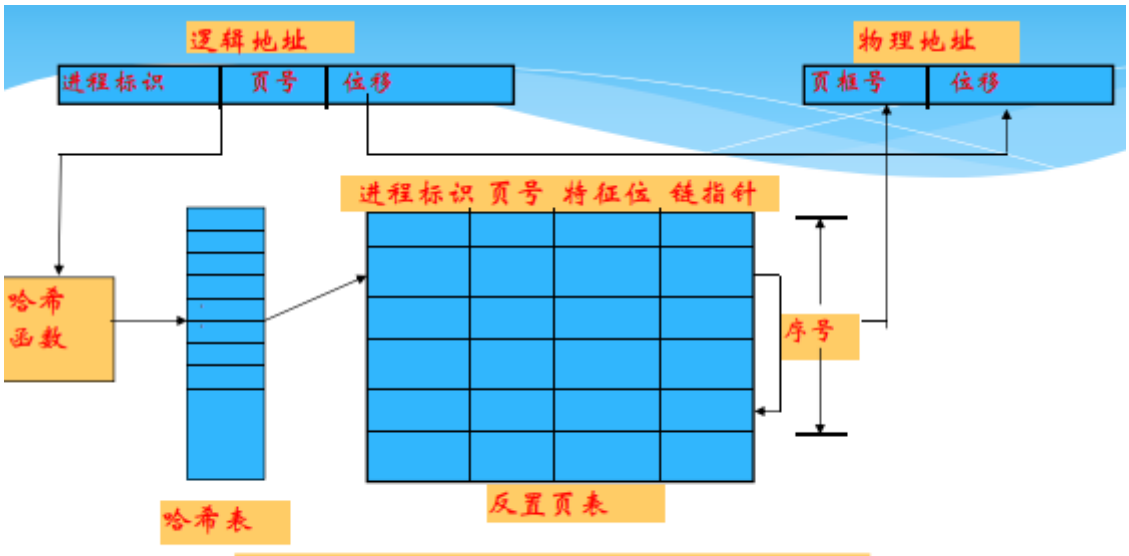
设备管理

- 作业调度与进程调度的关系



- 什么是虚拟设备
  - 使用一类物理设备模拟另一类物理设备的技术
  - 通常是使用共享型外围设备模拟独占型外围设备
- IO控制方式
  - 程序控制I/O (轮询方式)
    - 处理器代表给I/O模块发送一个I/O命令，该进程进入忙式等待 (busy-waiting)，等待操作的完成，然后才可以继续操作
  - 中断驱动 I/O
    - 处理器代表进程向I/O模块发出一个I/O命令，然后继续执行后续指令，当I/O模块完成工作后，处理器被该模块中断
    - 如果该进程不需要等待I/O完成，则后续指令可以仍是该进程中的指令，否则，该进程在这个中断上挂起，处理器执行其他工作
  - 直接存储器访问(DMA)
    - 一个DMA模块控制主存和I/O模块之间的数据交换。为传送一块数据，处理器给DMA模块发请求，只有当整个数据块传送结束后， 处理器才被中断

反置页表

- 页表设计的一个**重要缺陷是页表的大小与虚拟地址空间的大小成正比**
- 在反向页表方法中，**虚拟地址的页号部分使用一个简单散列函数映射到哈希表中。**哈希表包含一个指向反向表的指针，而反向表中含有页表项
- 通过这个结构，哈希表和反向表中只有一项对应于一个**实存 页(面向实存)**，而不是虚拟页(面向虚存)
- 因此，不论由多少进程、支持多少虚拟页，页表都只需要实 存中的一个固定部分
- 

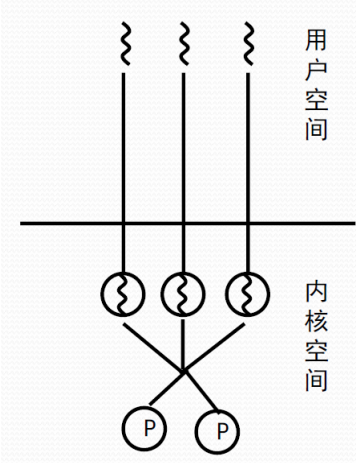
反置页表及其地址转换

传统Unix系统调度

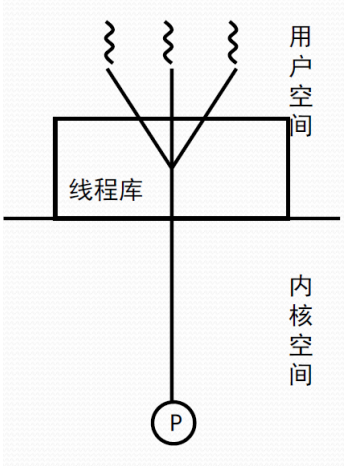
- 多级反馈队列，每个优先级队列使用时间片轮转
- 每秒重新计算每个进程的优先级
- 给每个进程赋予基本优先级的目的是把所有进程划分成固定的优先级区
- 可控调节因子

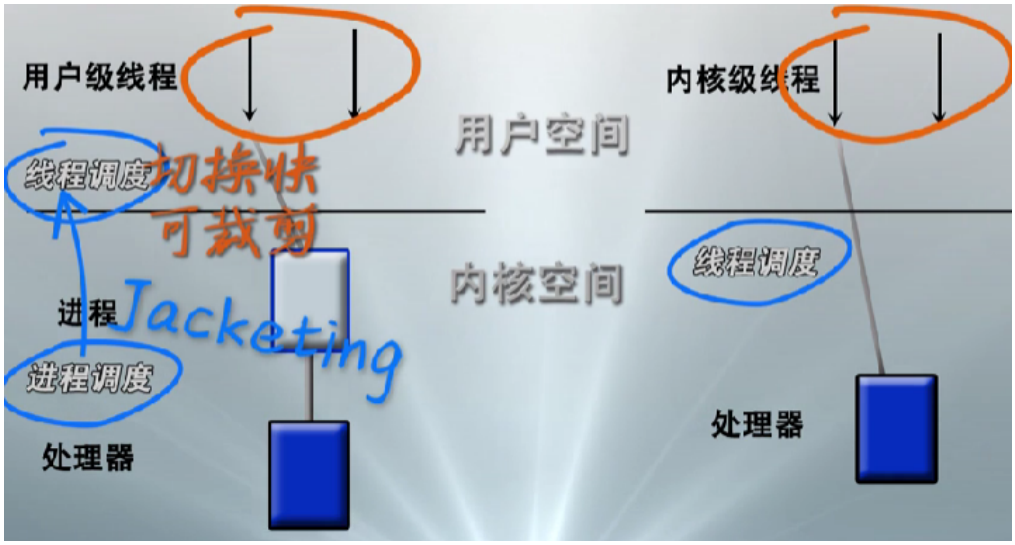
用户级线程和内核级线程

- 内核级线程
  - 线程的所有工作由OS内核来做，并提供了一个应用程序设计接口API，供开发者使用内核级进程（KLT）
  - OS 直接调度 KLT，KLT 用于**解决物理并行性问题**，内核可以感知到所有的内核级线程，可以控制器其数据结构，内核调度在线程的基础上进行
  - 示意图：



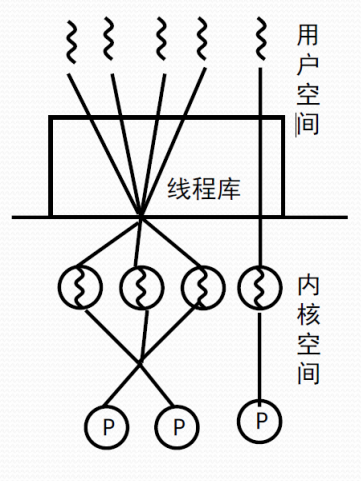
  - 内核级线程的特点
    - 优点：
      1. 在多处理器上内核可以同时调度同一进程的多个线程运行。
      2. 进程中的某一线程被阻塞了，内核能调度同一进程的其它线程占有处理器运行，也可以运行其他进程。
      3. 由于内核比较小，内核自身也可用多线程技术实现，能提高操作系统的执行速度和效率。
      4. 能够解决物理并行性问题。
    - 缺陷：
      1. 应用程序线程在用户态运行，线程调度和管理在内核实现，在同一进程中，控制权从一个线程传送到另一个线程时需要模式切换，系统开销较大。
      2. 线程调度开销大，线程通信开销小。
- 用户级线程
  - 用户空间运行的线程库，提供多线程应用程序的开发和运行支撑环境。任何应用程序均需通过线程库进行程序设计，再与线程库连接后运行。
  - 线程管理的所有工作都由**应用程序**完成，**内核没有**感知到线程的存在，内核感知到的单位是进程。
  - 示意图



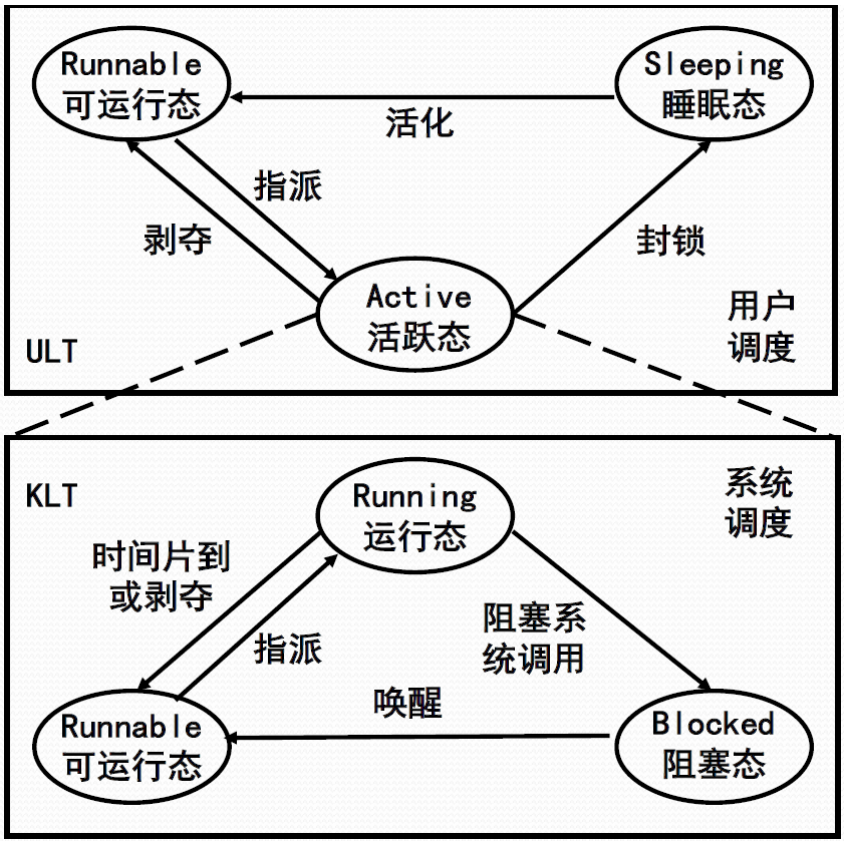
- 用户级线程的特点
  - 优点：
    1. 节省开销和内核资源：所有线程管理数据结构均在进程的用户空间中，线程切换不需要内核模式，能**节省模式切换开销和内核的宝贵资源。**
    2. 允许进程按应用特定需要选择调度算法，甚至根据应用需求裁剪调度算法。
    3. 可移植性好：能运行在任何 OS 上，内核在支持 ULT 方面不需要做任何工作。
    4. ULT 可以解决逻辑并行性问题。
  - 缺点：
    1. 不能利用多处理器的优点，OS 调度进程，仅有一个 ULT 能执行。
    2. 一个 ULT 的阻塞，将引起整个进程的阻塞：不能完成切换线程，因为内核感知不到进程中的线程的存在。（Jacketing技术）
    3. ULT 不能解决物理并行性问题。
- Jacketing技术
  - 作用：把阻塞式的系统调用改造为非阻塞式的——来解决一个ULT的阻塞导致整个进程的阻塞；避免程序因此从运行态转换为阻塞态（转换频繁开销大）
  - 当线程陷入系统调用时，执行 Jacketing 程序。
  - 由 Jacketing 程序来检查资源使用情况，以决定是否执行**进程切换**或**传递控制权给另一个线程**
  - 

- 受限条件下的并行：受限条件是同步关系（等待）

- 多线程实现的混合策略



- 策略：
  1. 在用户空间完成所有的线程的创建工作
  2. 单应用的多个 ULT 可以**映射**成一些 KLT，通过调整 KLT 数目，可以达到较好的并行效果
  3. 在混合式线程中，内核必须支持内核级多线程的建立、调度和管理，同时也允许应用程序建立、调度和管理用户级线程。
  4. ULT 的切换仅在用户空间中且仅需要少量机器指令，而 KLT 需要用户态到内核态到用户态的完整上下文切换，修改内存映像，使得高速缓存失效，导致数量级的延迟。
- 特点
  1. 合并了用户级线程/内核级线程设施
  2. **线程创建完全在用户空间中完成，线程的调度和同步也在应用程序中进行**
  3. 一个应用中的多个用户级线程被**映射到一些**（小于等于用户级线程数目）内核级线程上
  4. 程序员可以针对特定应用和机器调节内核级线程的数目，以达到整体最佳结果
  5. 该方法将会结合纯粹用户级线程方法和内核级线程方法的优点，同时减少它们的缺点
- 混合策略下的线程状态



  - KLT 的三态模型，由系统调度负责
  - ULT 的三态模型，由用户调度负责
  - **活跃态的 ULT 代表绑定 KLT 的三态**

        - 活跃态的 ULT 运行时可激活用户调度，非阻塞系统调用可使用 Jacketing 启动用户调度，调整活跃态 ULT

进程与线程

- 操作系统中引入进程的目的是为了使多个程序并发执行，以改善资源使用率和提高系统 效率
- 操作系统中再引入线程，则是为了减少程序并发执行时所付出的时空开销，使得并发粒度更细、并发性更好

特权指令与处理器状态

- 从资源管理和控制程序执行的角度出发，必须设置 特权指令，提供给操作系统的核心程序使用
- 处理器状态
    - 管理状态(特权状态、系统模式、特态或管态): 处 理器可以执行全部指令，使用所有资源，并具有 改变处理器状态的能力
    - 用户状态(目标状态、用户模式、常态或目态): 处 理器只能执行非特权指令
    - 核心状态、管理状态和用户状态

死锁产生的四个必要条件

1. **互斥条件**: 进程应互斥使用资源，任一时刻一个资源仅为一个进程独占
2. **占有和等待条件**: 一个进程请求资源得不到满足而等待时，不释放已占有的资源
3. **不剥夺条件**: 任一进程不能从另一进程那里抢夺资源
4. **循环等待条件**: 存在一个循环等待链，每一个进程分别等待它前一个进程所持有的资源
5. 前三个是死锁存在的必要条件，但不是充分条件，第四个条件是前三个条件同时存在时所产生的结果